PROF. DR.-ING. MARK SCHUTERA

_SUPERVISED LEARNING

UNFINISHED LECTURE NOTES

Copyright © 2026 Prof. Dr.-Ing. Mark Schutera

PUBLISHED BY UNFINISHED LECTURE NOTES

Combobulated with the help of multiple large language model driven tools. Licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("CC BY-NC-SA 4.0"). You may not use this file for commercial purposes. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original You must obtain explicit permission from the author for uses beyond those permitted by this license. To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/. Unless required by applicable law or agreed to in writing, distributed material is provided on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the license for details.

Last updated, April 2026 — v0.0.6

"IN MY DEFENSE, I WAS LEFT UNSUPERVISED".

DER ZAUBERLEHRLING

Contents

I Geometry of Pattern 11

Density-Based Clustering 13

Dimensionality Reduction 29

Index 41

Introduction

Supervised learning is limited to tasks where we have labeled data and clear objectives. Beyond this limited context of learning, we fall back to the pattern in the data itself.

We explore how pattern emerges, from other sources of information beyond explicit labels. We will see how pattern can be discovered in data, represented, leveraged and utilized. In that sense this course is about any form of learning that is not supervised learning.

Each lecture follows a consistent pattern. To start of the contents of the lecture will be motivated by a narrative, a theoretical primer, and some sort of hands-on experience to build intuition. Then we will dive into the, theory and algorithms, then reflection. Theory sessions are 45 minutes, followed by 45 minutes of coding practice to enforce reflection and deepen understanding of the concepts.

A Philosophy of Education

The beautiful risk of education is that it is an encounter. I am trying to model a way of thinking, and reasoning, then stepping back. The rest is yours.

- **Start with why.** A method understood only procedurally is a method you will misapply when it matters. Before learning how something works, understand why it was built, what it replaced, and where it fails. The mechanics then follow from necessity, not memorization.
- **Build taste alongside competence.** You need a sense of what beautiful is, you need to train your aesthetics before you can reliably produce it. This develops by studying what others have built, making it your own, then making your own, and honestly evaluating the gap between them. Taste is the compass. Skill is the engine. You need both, in that order.
- **Failure is the designed medium.** Each attempt teaches what a clean run cannot. The insight compounds across tries, not from a single correct execution. Those who never start failing will fail.
- **Work in the open.** Showing half-finished thinking, naming mistakes as they happen, is a professional skill. We practice it here. It is how knowledge actually moves between people. And it is how knowledge moves towards you, at unprecedented speed.
- **The fallacy of mediocracy.** Iteration and failure are the method, not the excuse for settling. Do not confuse velocity with quality. Whatever you learn, whatever you build: make it good.
- **Outcomes emerge. They are not delivered.** Judgment, taste, the capacity to sit with uncertainty: these are not course outputs. They are what you build from the encounter. I can create the conditions. The rest depends on what you do with them.

Part I

Geometry of Pattern

Density-Based Clustering

It is all about that space, about that space.

The Why

The clustering algorithms we have seen so far share a common mechanism of assigning points to clusters based on their distance to an other point; a centroid or a linkage point. Every point, no matter how isolated, is absorbed into some cluster latest at the final merge. Real data contains noise: GPS measurements with dropouts, sensor readings during interference, survey responses that are genuine outliers.

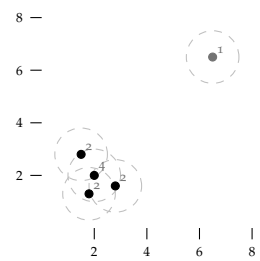


Figure 1: Each point carries its own dashed ε-neighbourhood; the small grey digit next to a point gives the number of points inside that circle, itself included. In the dense cluster at the lower left the four neighbourhoods overlap and chain the points together; the isolated p_n at the upper right has no companions within ε. Density, not distance to a centre, determines membership.

No centroid required. A density-based cluster is a space, not a mean and not a link. Its shape is implicit in the data, seeded by the points themselves; nothing forces it to be convex or spherical; detects noise, outliers and sparse bridges naturally, and avoids committing to K in advance. This geometric-assumption-free view is both the strength of density-based methods and, as we will see, the source of their own limitations.

In order to move from pairwise distances to density-aware neighbourhoods we have good reason to find ways to,

- define what it means for a region of space to be dense or sparse.
- grow clusters of arbitrary shape, including non-convex shapes, from local density, without assuming convex or spherical structure.
- separate signal from noise, so that sparse outliers are labelled as such rather than forced into a cluster.
- let the data itself decide how many clusters exist, without committing to K in advance.

Hands On Experience

CONSIDER the same six canonical points, now with a seventh point $x_7{=}(4,5)$ placed between the two natural groups.

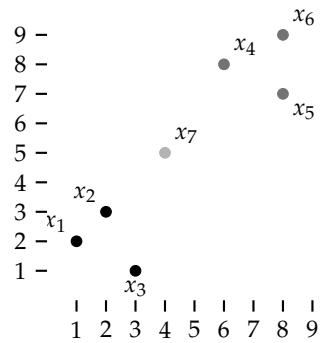


Figure 2: Six canonical points plus $x_7{=}(4,5)$ sitting between the two natural groups. Which points lie in dense neighbourhoods, and which sits in a sparse region?

DRAW A CIRCLE of radius $\varepsilon{=}2.5$ around each point and count who sits inside. The three bottom-left points find each other, the three top-right points find each other, and x_7 finds empty space; read off the data without ever choosing a number of clusters.

THE NEIGHBOURHOOD RADIUS ε is the distance within which a point looks for companions. Too small and every point is left alone; too large and everything collapses into one cluster. The right value lives where the data transitions from dense to sparse, and we will read it off the data itself using the k-distance plot.

OUR APPROACHES SO FAR WOULD REFUSE to let x_7 be lonely. With $K{=}2$ it hands the point to whichever centroid happens to sit slightly closer, an assignment driven by initialisation rather than by any structure in the data. Hierarchical clustering would fold x_7 into some merge and bury it inside a cluster it never really joined.

DENSITY ALLOWS US TO DO SO. A point whose ε-neighbourhood is empty is not handed to the nearest cluster; it is labelled p_n. Cluster shape is read off local neighbourhoods with a sense for p_b's, the number of clusters emerges from where density breaks instead of from a chosen K, and sparse points are flagged instead of absorbed.

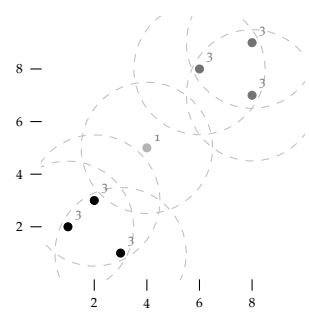


Figure 3: The seven canonical points with their dashed $\varepsilon{=}2.5$ neighbourhoods. The small grey digit next to each point gives $|N_\varepsilon|$, itself included. Each natural group forms a three-point neighbourhood; x_7 sits in a sparse region and finds only itself within ε.

THE LEARNING OBJECTIVES of this lecture:

- Trace a density clustering by hand on a small dataset and learn how to set the neighbourhood radius ε, and the minimum number of points $n_{\min}$ from the data itself.
- Understand the concepts of core (p_c), border (p_b), and noise (p_n) points by formalising neighbourhoods.
- Recognise when density-based clustering succeeds where K-Means and hierarchical fail, where they share characteristics, and where it fails in turn under varying density, and in high dimensionality.

The Density-Based Algorithm

DB SCAN[1] or Density-Based Spatial Clustering of Applications with Noise, builds clusters from local density rather than from a chosen number of centroids or a sequence of merges. Two parameters govern the algorithm: ε, the neighbourhood radius, and $n_{\min}$, the minimum number of points required to form a dense region.

[1] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 226–231. AAAI Press, 1996. https://scholar.google.com/scholar?q=Ester+Kriegel+Sander+Xu+1996+density-based+algorithm+discovering+clusters

GIVEN DATA $X = \{x_1, \ldots, x_n\}$ and parameters $(\varepsilon, n_{\min})$, DB Scan returns a labelling

$$\tilde{y} : X \longrightarrow \{1, 2, \ldots, K, \ p_n\}$$

where K is determined by the data rather than fixed in advance. The output is a flat partition plus a p_n set N.

EVERY POINT IS CLASSIFIED into one of three roles before any cluster is formed. The ε-neighbourhood of a point p is the set of all points within distance ε:

$$N_\varepsilon(p) = \{q \in X : d(p, q) \leq \varepsilon\}.$$

- A point p_c is a core point if $|N_\varepsilon(p_c)| \geq n_{\min}$.
- A point p_b is a border point if $|N_\varepsilon(p_b)| < n_{\min}$ but $p_b \in N_\varepsilon(p_c)$ for some core point p_c.
- A point p_n is noise if it is neither core nor border.

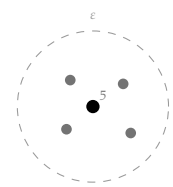


Figure 4:

CORE POINT. The black point has $|N_\varepsilon|$=5 $\geq n_{\min}$=4. Four companions sit inside its dashed ε-neighbourhood. The small grey digit at the upper right gives $|N_\varepsilon|$, the point itself included.

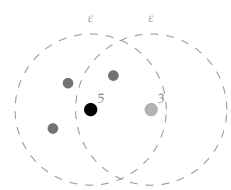


Figure 5:

BORDER POINT p_b. The light grey point on the right has $|N_\varepsilon|$=3 $< n_{\min}$=4. It is not a p_c itself, but it falls inside the ε-neighbourhood of the black p_c on the left and is absorbed into that cluster.

DENSITY REACHABILITY is not symmetric. A p_b is reachable from a p_c whenever $p_b \in N_\varepsilon(p_c)$, but p_c is not reachable from p_b because p_b is not a p_c. Cluster membership propagates from p_c's outward, never in reverse.

DENSITY CONNECTIVITY extends reachability to full cluster membership. Point q is density-reachable from p if there exists a chain of points $p{=}p_1, p_2, \ldots, p_m{=}q$ such that

$$p_{i+1} \in N_\varepsilon(p_i) \quad \text{and} \quad |N_\varepsilon(p_i)| \geq n_{\min} \quad \text{for } i = 1, \ldots, m-1.$$

Two points p, q are density-connected if there exists a third point o_c from which both are density-reachable; a cluster is then defined as a maximal set of pairwise density-connected points.

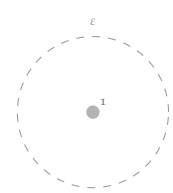


Figure 6:

NOISE POINT p_n. With $|N_\varepsilon|$=1 and no p_c within ε, the point belongs to no cluster. The dashed circle is otherwise empty.

TRACE THROUGH OUR SEVEN POINTS. Dashed circles are ε=2.5 neighbourhoods; dark points belong to C_1, grey to C_2, and an open circle marks p_n.

INITIALIZATION is trivial as the set of clusters and their core-memberships is the same regardless of the order in which points are processed. Any unvisited p_c triggers a new cluster; absorbing surrounding p_b's immediately; and growing the cluster by absorbing other p_c's and then their p_b's. Only p_b's that happen to lie in the ε-neighbourhood of several p_c's can be assigned to different clusters depending on visit order, but keeping their point class p_c, p_b, or p_n.

Algorithm 1: Naive Density-Based Clustering (DBSCAN)

Require: Data X, neighbourhood radius ε, density threshold $n_{\min}$
Ensure: Cluster label for each point (or p_n)

```
Mark all points as unvisited; C ← 0
for each unvisited point p_i ∈ X do
    Mark p_i as visited
    Compute N ← N_ε(p_i)
    if |N| < n_min then
        Label p_i as p_n
    else
        Label p_i as p_c; C ← C + 1; assign p_i to cluster C
        for each p_j ∈ N \ {p_i} do
            if p_j is unvisited then
                Mark p_j as visited
                Compute N′ ← N_ε(p_j)
                if |N′| ≥ n_min then label p_j as p_c; N ← N ∪ N′
                end if
            end if
            if p_j has no cluster then assign p_j to C (as p_c or p_b)
            end if
        end for
    end if
end for
```

TERMINATION AND DETERMINISM. The outer loop visits every point at most once and the inner loop only grows by adding previously unvisited points, so the algorithm halts after $O(n)$ neighbourhood queries. The resulting cluster set is determined uniquely by the data and the pair $(\varepsilon, n_{\min})$: p_c's are fixed by the $|N_\varepsilon|$ threshold.

COMPLEXITY. Each of the n points triggers one ε-neighbourhood query costing $O(n)$, giving $O(n^2)$ overall. A spatial index (k-d or ball tree) reduces each query to $O(\log n)$, yielding $O(n \log n)$ in low dimensions.

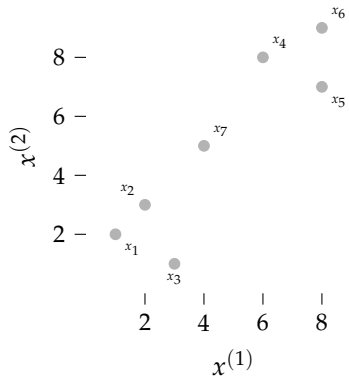


Figure 7:

INITIALIZATION: all n=7 points unvisited. Parameters ε=2.5, $n_{\min}$=2 are fixed. No seed necessary. Randomly picking x_1 as the first point to process.

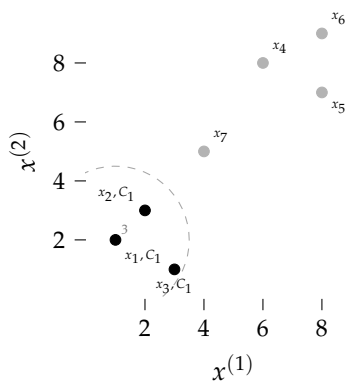


Figure 8:

VISIT x_1: $N_{2.5}(x_1)=\{x_1,\ x_2,\ x_3\}$, $|N|=3 \geq 2$, so x_1 is a p_c.

Seed cluster C_1; assign x_1 to C_1 and add x_2 and x_3 to the expansion queue. Points outside the dashed circle remain unvisited.

Distances: $d(x_1, x_2)=\sqrt{2}\approx 1.41$ ✓, $d(x_1, x_3)=\sqrt{5}\approx 2.24$ ✓.

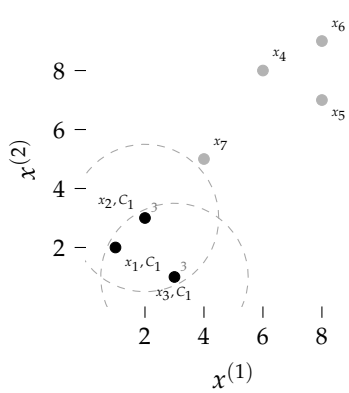


Figure 9:

EXPAND C_1: dequeue x_2: $N_{2.5}(x_2)=\{x_1, x_2, x_3\}$, all already assigned to C_1, no new points added.

Dequeue x_3: $N_{2.5}(x_3)=\{x_1, x_2, x_3\}$, same result.

Expansion queue empty: $C_1=\{x_1,\ x_2,\ x_3\}$ is sealed.

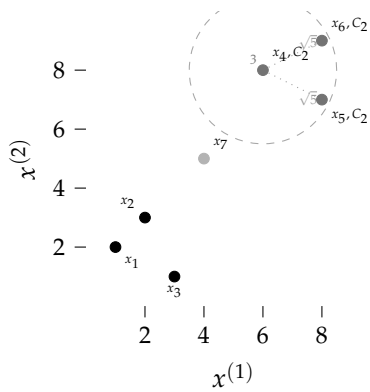


Figure 10:

VISIT x_4: $N_{2.5}(x_4)=\{x_4,\ x_5,\ x_6\}$, $|N|=3 \geq 2$, so x_4 is a p_c.

Seed C_2; expand from x_5 and x_6: both are p_c's and their neighbourhoods cover only $\{x_4, x_5, x_6\}$.

Distances: $d(x_4, x_5)=\sqrt{5}\approx 2.24$ ✓, $d(x_4, x_6)=\sqrt{5}\approx 2.24$ ✓, $d(x_4, x_7)=\sqrt{13}\approx 3.61$ ×.

$C_2=\{x_4,\ x_5,\ x_6\}$ sealed. Only x_7 remains unvisited.

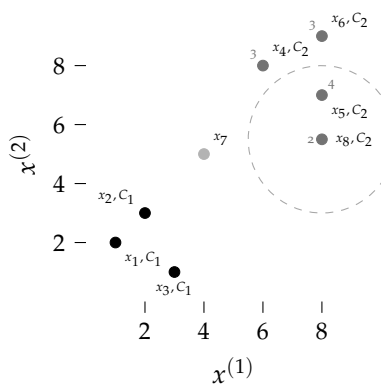


Figure 11:

HYPOTHETICAL POINT $x_8=(8,\ 5.5)$: $d(x_5, x_8)=1.5 \leq \varepsilon$, so x_5 absorbs x_8 into its neighbourhood.

$N_{2.5}(x_8)=\{x_8,\ x_5\}$, so $|N|=2 < n_{\min}=3$; x_8 is a border point. Because x_5 is already a core point of C_2, x_8 is classified as C_2.

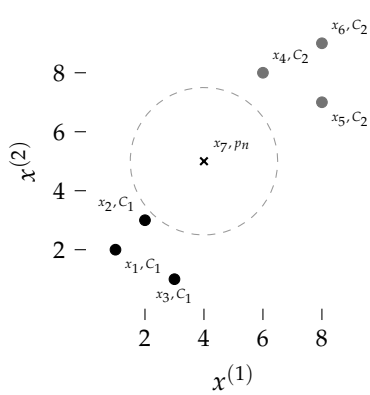


Figure 12:

VISIT x_7: its two nearest neighbours are x_2 at $d\approx 2.83$ and x_4 at $d\approx 3.61$, both beyond $\varepsilon=2.5$. So $N_{2.5}(x_7)=\{x_7\}$, $|N|=1 < 2$, and x_7 is labelled p_n.

All points visited. Final result: $C_1=\{x_1, x_2, x_3\}$, $C_2=\{x_4, x_5, x_6\}$, and x_7 left as noise. No K was chosen; the noise point was never forced into a cluster.

Hyperparameter Selection

CHOOSING $n_{\min}$ follows a practical rule of thumb: set $n_{\min} > d+1$ where d is the dimension of the feature space. For two-dimensional data, $n_{\min}=3$ or 4 is typical; higher dimensions require larger values to keep sparse neighbourhoods from being treated as dense.

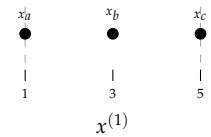


Figure 13: How many points do you need at least to fence in a point in 2D? 3D?

CHOOSING ε uses the k-distance plot: for each point, compute the distance $d_k(x_i)$ to its k-th nearest neighbour at $k=n_{\min}-1$. The $(n_{\min}-1)$-th nearest neighbour distance is the tightest radius that still qualifies.

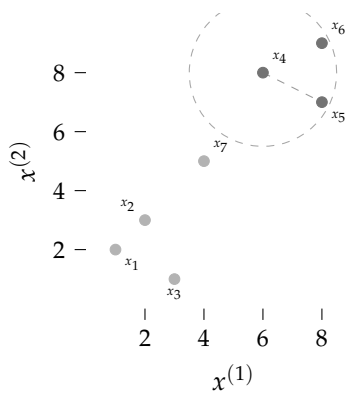


Figure 14:

k-TH NEAREST NEIGHBOUR OF x_4: $n_{\min}=2$, $k=n_{\min}-1=1$. The dashed circle shows the $\varepsilon=2.5$ neighbourhood; the nearest neighbour of x_4 is x_5 at distance $d_k(x_4)=\sqrt{5}\approx 2.24$, well inside ε.

and sort these distances in ascending order,

$$d_k^{(1)} \leq d_k^{(2)} \leq \ldots \leq d_k^{(n)}.$$

SENSITIVITY SWEEP. Try a few $n_{\min}$ values ($d+1$, $2d$, etc.) and compare the resulting k-distance plots. The value that gives the sharpest, most unambiguous elbow is usually the right one.

The curve rises slowly in dense regions and sharply in sparse ones; the elbow marks the transition and is the recommended ε. Notice, that the sorting does not reflect cluster membership: points from different clusters can be interleaved in the k-distance plot.

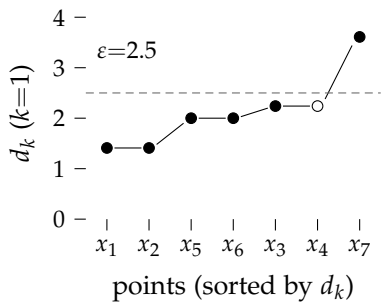
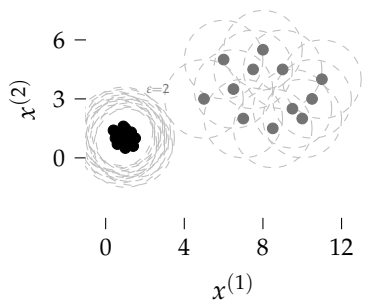


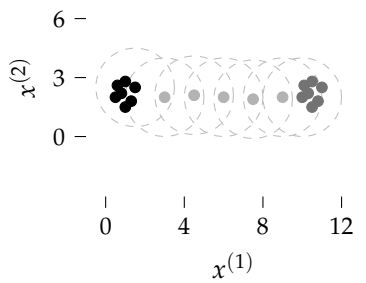
Figure 15: k-distance plot for the seven canonical points with $k{=}n_{\min}{-}1{=}1$. Each $d_k^{(i)}$ is the distance from one point to its nearest neighbour, sorted ascending. The first six values sit at $\sqrt{2}{\approx}1.41$, 2.00, or $\sqrt{5}{\approx}2.24$; $d_k^{(7)}$ jumps to $\sqrt{13}{\approx}3.61$ (the lone x_7). The unfilled dot marks the elbow, and any ε in the gap, such as $\varepsilon{=}2.5$ (dashed), separates the dense six from the sparse outlier.

When DB Scan Fails

DB Scan has its own pathologies. Every density-based method sets a single threshold. The most common arise from the single global ε, which filters or bridges. Then we have the curse of dimensionality, which makes all points look far apart and destroys the notion of density.



Varying Density
Figure 16: Varying cluster density. An ε tight enough to resolve the left cluster leaves the right cluster disconnected; an ε large enough to connect the right cluster merges everything on the left into one blob, loosing expressiveness of the clustering. No single global threshold works for both.



Bridging and Chaining
Figure 17: Bridging: a thin chain of p_c-eligible points (light) spans the gap between two well-separated clusters. The overlapping ε-circles show how density reachability chains through p_c's, fusing both into one cluster. Increasing $n_{\min}$ or decreasing ε breaks the bridge at the cost of shrinking the clusters themselves.

OPTICS or Ordering Points To Identify the Clustering Structure [2] introduces two per-point distances. The core distance ε_c is the per-

[2] M. Ankerst, M. M. Breunig, H.-P. Kriegel, and J. Sander. OPTICS: Ordering points to identify the clustering structure. In *Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data*, pages 49–60, 1999. DOI: 10.1145/304182.304187. https://scholar.google.com/scholar?q=Ankerst+Breunig+Kriegel+Sander+1999+OPTICS+ordering+points

point analogue of the global ε, the smallest radius that captures $n_{\min}$ neighbours:

$$\varepsilon_c(x_i) = \min\{r : |N_r(x_i)| \geq n_{\min}\}.$$

The reachability distance $r(x_i, x_j)$ is defined for every pair where x_i is an already-visited p_c and x_j is an unvisited point. It measures the cost for x_i to absorb x_j, and is at least $\varepsilon_c(x_i)$, even if x_j sits closer:

$$r(x_i, x_j) = \max\{\varepsilon_c(x_i),\ d(x_i, x_j)\}.$$

Dense regions have small ε_c, sparse regions large ones. The max ensures that a point is never reached at a distance shorter than its ε_c, so reachability adapts to local density automatically.

HDBSCAN or Hierarchical DBSCAN [3] replaces the Euclidean distance with a mutual reachability distance:

$$d_{\text{mreach}}(x_i, x_j) = \max\{\varepsilon_c(x_i),\ \varepsilon_c(x_j),\ d(x_i, x_j)\}.$$

This spreads apart points in sparse regions; exactly those that should not be merged early. From here HDBSCAN builds a full density hierarchy by applying single-linkage clustering to the mutual reachability distance, and potentially condenses it into a flat clustering by selecting the most persistent clusters across all density levels.

[3] R. J. G. B. Campello, D. Moulavi, and J. Sander. Density-based clustering based on hierarchical density estimates. In *Advances in Knowledge Discovery and Data Mining (PAKDD)*, pages 160–172, 2013. https://scholar.google.com/scholar?q=Campello+Moulavi+Sander+2013+density-based+clustering+hierarchical+density+estimates

THE ONLY PARAMETER the user must set is $n_{\min}$, and the result is often robust to its choice.

DENSITY HIERARCHIES AND DENDROGRAMS. The density language and the hierarchical-clustering language describe the same structure from different angles.

THE CURSE OF DIMENSIONALITY: All distance-based methods share a deeper vulnerability as the number of features d grows, the volume of the space increases so fast that a fixed number of points N becomes sparse everywhere.

CONSIDER N POINTS drawn uniformly in the unit hypercube $[0,1]^d$. The mean nearest-neighbour distance scales as

$$d_{\text{NN}} \ \propto\ N^{-1/d}.$$

Fix $N{=}10$ and watch the spacing grow with d:

$$\begin{aligned} d = 1: &\quad 10^{-1/1} = 0.10 \\ d = 2: &\quad 10^{-1/2} \approx 0.32 \\ d = 3: &\quad 10^{-1/3} \approx 0.46 \\ d = 10: &\quad 10^{-1/10} \approx 0.79 \\ d = 50: &\quad 10^{-1/50} \approx 0.96 \end{aligned}$$

By $d{=}50$ every point is nearly as far from its neighbour as from the boundary of the cube. The points have not moved apart; the space has grown underneath them.

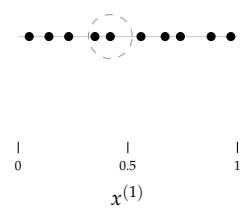


Figure 18: $N{=}10$ points in $[0,1]^1$: mean NN spacing $\approx 10^{-1}{=}0.1$. The line is well covered.

$\propto$ means "proportional to": $a \propto b$ says $a = c \cdot b$ for some constant c independent of the variables of interest.

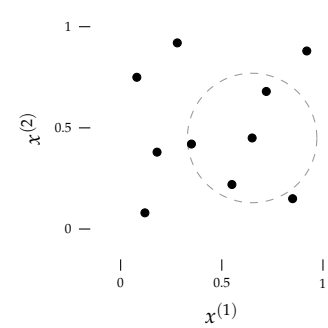


Figure 19: Same $N{=}10$ points in $[0,1]^2$: the space has grown from a line to a square, and the mean NN spacing rises to $10^{-1/2}{\approx}0.32$.

For distance-based methods this is fatal. The ε-neighbourhood of a point captures a hypersphere whose volume shrinks relative to the cube: $V_{\text{sphere}} / V_{\text{cube}} \to 0$ as d grows. A radius that includes many neighbours in two dimensions encloses almost nothing in fifty. Either ε must grow until every point is a neighbour, collapsing all structure into one cluster, or it stays small and every point is labelled p_n.

Examples & Exercises

COMPUTING BY HAND reveals what the algorithm actually does at each step; run it slowly and feel where density propagates and where it stops.

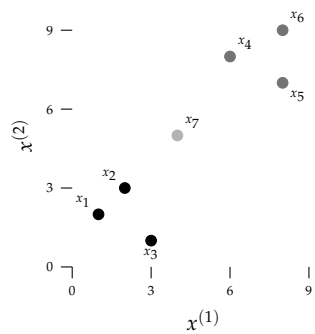


Figure 20: Seven points for the DBSCAN trace. x_7=(4, 5) sits between the two natural groups.

GIVEN SEVEN POINTS x_1=(1, 2), x_2=(2, 3), x_3=(3, 1), x_4=(6, 8), x_5=(8, 7), x_6=(8, 9), x_7=(4, 5), run DBSCAN with ε=2.5 and $n_{\min}$=2. Your tasks: compute the neighbourhood size $|N_{2.5}(x_i)|$ for each point, classify each point as p_c, p_b, or p_n, and identify the clusters.

NEIGHBOURHOOD SIZES. For each point x_i count all points within distance 2.5 (including x_i itself). Here is the full calculation for x_1:

$$
\begin{aligned}
d(x_1, x_2) &= \sqrt{(2-1)^2 + (3-2)^2} \\
&= \sqrt{1+1} \\
&= \sqrt{2} \\
&\approx 1.41 \quad (\leq 2.5, \text{ counted}) \\
d(x_1, x_3) &= \sqrt{(3-1)^2 + (1-2)^2} \\
&= \sqrt{4+1} \\
&= \sqrt{5} \\
&\approx 2.24 \quad (\leq 2.5, \text{ counted}) \\
d(x_1, x_7) &= \sqrt{(4-1)^2 + (5-2)^2} \\
&= \sqrt{9+9} \\
&= \sqrt{18} \\
&\approx 4.24 \quad (> 2.5, \text{ not counted})
\end{aligned}
$$

So $N_{2.5}(x_1) = \{x_1, x_2, x_3\}$ and $|N_{2.5}(x_1)| = 3 \geq 2$: x_1 is a p_c. Compute the remaining neighbourhood sizes yourself, then verify against the table below.

VERIFY x_7 IS p_n. Compute $d(x_7, x_3)$ to confirm that x_7 lies outside

Point	Coords	$\lvert N_{2.5}\rvert$	Type	Neighbours in $N_{2.5}$	Cluster
x_1	$(1,2)$	3	p_c	$\{x_1, x_2, x_3\}$	C_1
x_2	$(2,3)$	3	p_c	$\{x_1, x_2, x_3\}$	C_1
x_3	$(3,1)$	3	p_c	$\{x_1, x_2, x_3\}$	C_1
x_4	$(6,8)$	3	p_c	$\{x_4, x_5, x_6\}$	C_2
x_5	$(8,7)$	3	p_c	$\{x_4, x_5, x_6\}$	C_2
x_6	$(8,9)$	3	p_c	$\{x_4, x_5, x_6\}$	C_2
x_7	$(4,5)$	1	p_n	$\{x_7\}$ only	—

Table 1: DBSCAN trace for ε=2.5, $n_{\min}$=2. All six canonical points are p_c's; x_7 has no neighbour within radius 2.5 and is labelled p_n. The two natural clusters are recovered without specifying K and without assigning p_n to either group.

the ε-neighbourhood of its nearest candidate:

$$\begin{aligned} d(x_7, x_3) &= \sqrt{(4-3)^2 + (5-1)^2} \\ &= \sqrt{1+16} \\ &= \sqrt{17} \\ &\approx 4.12 \quad (> 2.5) \end{aligned}$$

Compute $d(x_7, x_4)$ yourself. Why is x_7 correctly identified as p_n rather than being assigned to whichever cluster is nearer?

Adding a border point. Place an eighth point x_8=(8, 5.5) into the dataset and raise the density threshold to $n_{\min}$=3. Compute $\lvert N_{2.5}(x_8)\rvert$ and classify x_8 as p_c, p_b, or p_n. Which cluster, if any, does x_8 join, and why? Verify that the six original cluster members remain p_c's under the new threshold.

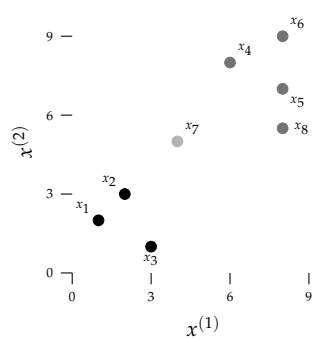


Figure 21: Eight points for the border-point exercise. x_8=(8, 5.5) sits below C_2.

Density reachability from x_8. Still using the eight-point dataset with ε=2.5 and $n_{\min}$=3: is x_8 density-reachable from x_4? If so, write down the chain of points that connects them. Now reverse the question: is x_4 density-reachable from x_8? Why or why not?

Parameter sensitivity. Re-run the trace with ε=4.5 and $n_{\min}$=2. Does x_7 change classification? At what minimum value of ε does x_7 first enter the neighbourhood of some p_c?

Building the k-distance plot. Return to the original seven points with $n_{\min}$=2, so k=$n_{\min}$−1=1. For each point x_i, compute

the distance to its nearest neighbour $d_1(x_i)$. Here is x_5:

$$\begin{aligned} d(x_5,\, x_4) &= \sqrt{(8-6)^2 + (7-8)^2} \\ &= \sqrt{5} \\ &\approx 2.24 \\ d(x_5,\, x_6) &= \sqrt{(8-8)^2 + (9-7)^2} \\ &= \sqrt{4} \\ &= 2.00 \\ d_1(x_5) &= 2.00 \quad \text{(nearest neighbour is } x_6\text{)} \end{aligned}$$

Compute $d_1(x_i)$ for the remaining six points. Sort all seven values in ascending order and sketch the curve. Where is the elbow? Does the ε=2.5 from the earlier trace sit in the gap?

Comparison with K-Means. Run K-Means with K=2 on the same seven points. Which cluster does K-Means assign x_7 to? Is that assignment more or less informative than DBSCAN's p_n label, and why?

K-Means baseline result. After convergence with K=2, the centroids settle at $\mu_1 \approx (2.0,\, 2.0)$ and $\mu_2 \approx (7.3,\, 8.0)$. Point x_7=$(4, 5)$ is closer to μ_1:

$$\begin{aligned} d(x_7,\, \mu_1) &= \sqrt{(4-2)^2 + (5-2)^2} \\ &= \sqrt{4+9} \\ &= \sqrt{13} \approx 3.61 \\ d(x_7,\, \mu_2) &= \sqrt{(4-7.3)^2 + (5-8.0)^2} \\ &= \sqrt{10.89+9} \\ &= \sqrt{19.89} \approx 4.46 \end{aligned}$$

K-Means assigns x_7 to C_1, as shown below. The assignment is not wrong in a strict sense; it is simply forced. K-Means has no p_n concept and cannot abstain.

From global to local density. Return to the eight-point dataset ($x_1, \ldots, x_7$ plus x_8=(8, 5.5)) with $n_{\min}$=3. For each point x_i, compute the core distance $\varepsilon_c(x_i)$: the smallest radius that captures at least $n_{\min}$ points, itself included. Here is x_5:

The core distance $\varepsilon_c(x_5)$=2.00 is the radius needed to capture $\{x_8,\, x_6,\, x_5\}$: exactly three points.

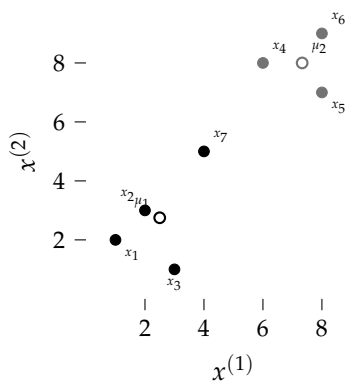


Figure 22: K-Means baseline with K=2. o-marks for centroids μ_1 and μ_2. Because K-Means must assign every point, x_7 is forced into C_1 (dark). Compare with Figure 12: DBSCAN labels x_7 as p_n and makes no forced assignment.

$$
\begin{aligned}
d(x_5,\, x_8) &= 1.50 \\
d(x_5,\, x_6) &= 2.00 \\
d(x_5,\, x_4) &= \sqrt{5} \\
&\approx 2.24 \\
\varepsilon_c(x_5) &= 2.00
\end{aligned}
$$

Compute ε_c for x_1 through x_4, x_6, x_7, and x_8. Which points have the largest core distance, and what does that tell you about their local density?

Now compute the mutual reachability distance for two pairs:

$$
\begin{aligned}
d_{\text{mreach}}(x_4,\, x_5) &= \max\{\varepsilon_c(x_4),\ \varepsilon_c(x_5),\ d(x_4,\, x_5)\} \\
d_{\text{mreach}}(x_7,\, x_2) &= \max\{\varepsilon_c(x_7),\ \varepsilon_c(x_2),\ d(x_7,\, x_2)\}
\end{aligned}
$$

Compare each to the raw Euclidean distance. For which pair does d_{mreach} differ from d, and why? Why does HDBSCAN not need a global ε?

THE CURSE OF DIMENSIONALITY BY THE NUMBERS. The mean nearest-neighbour spacing scales as $d_{\text{NN}} \propto N^{-1/d}$. Compute $N^{-1/d}$ for N=100 at $d = 2,\ 10,\ 50,\ 100$. At what dimension d does the spacing first exceed 0.9? If you tried to run DBSCAN at that dimension, what would happen to the ε-neighbourhoods?

BACK AT THE COFFEETABLE CODE. Same 1 797 aroma fingerprints, two coordinates. Implement epsilon-neighborhoods and the core-point classifier, use the k-distance plot to pick ε, then compare DBSCAN with K-Means and Ward at the same cluster count.

CODE will be provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

WHAT TO OBSERVE. Core points fill the dense hearts of each variety;

noise points land in the gaps no method should claim. DBSCAN follows density contours rather than drawing spherical cuts, so it agrees with K-Means in the cores but diverges at irregular boundaries. A single ε cannot capture clusters of very different densities. In next steps, learn about sensitivity sweeps and HDBSCAN to address this.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- What does density reachability gain over distance-to-centroid? Which K-Means failures disappear, which new ones appear?
- Why is density reachability asymmetric, and what does that imply for the determinism of p_b labels?
- A p_b sits inside two p_c's from different clusters. Is its label unique? What does that say about what a DB Scan cluster really is?
- Why is bridging the density-based dual of single-linkage chaining, and why does every remedy cost something?
- The k-distance plot shows a straight line with no elbow. What does this tell you, and which method would you try next?
- What is the essential difference between an OPTICS reachability plot and an HDBSCAN density hierarchy?
- When would you prefer DB Scan over Ward's method even if both produce a sensible partition?

RECAP of Key Concepts:

- DB Scan clusters are maximal sets of density-connected points; no centroid, no pre-specified K
- Every point is p_c when $|N_\varepsilon| \geq n_{\min}$, p_b when inside a p_c's neighbourhood without being p_c itself, or p_n
- Density reachability is transitive along p_c's but not symmetric; p_b labels depend on visit order
- The partition is determined by $(X, \varepsilon, n_{\min})$ up to shared p_b's; $O(n)$ queries, no restarts
- The k-distance plot with $k{=}n_{\min}{-}1$, sorted ascending, chooses ε at the elbow

- Failure modes: varying density, where no global ε fits two scales, and high dimensionality, where distances concentrate
- Bridging fuses clusters through chains of p_c's, the density dual of single-linkage chaining
- OPTICS exposes all density scales via a reachability plot; HDBSCAN extracts the most persistent clusters from a full density hierarchy

Two ways out of the curse of dimensionality. Either we collect exponentially more data [4], which is rarely possible, or we reduce the dimension so that the samples we already have sit densely in the space we actually care about. The observation that makes the second route tractable is that real datasets almost never use their ambient dimensions fully: the meaningful variation lives on a much lower-dimensional subset, and the rest is redundancy, correlation, or noise. Finding that signal-carrying subset is what the next part of these notes is about.

[4] A. Halevy, P. Norvig, and F. Pereira. The unreasonable effectiveness of data. *IEEE Intelligent Systems*, 24(2): 8–12, 2009. DOI: 10.1109/MIS.2009.36. https://scholar.google.com/scholar?q=Halevy+Norvig+Pereira+2009+unreasonable+effectiveness+of+data

Up next. Part II, Representing Pattern, leaves the original feature space behind and learns to construct new coordinates. Chapter 5 opens with principal component analysis as the linear baseline. Only once a dataset is described in the dimensions that carry signal do the clustering methods of Part I regain their footing.

Teaser. The curse of dimensionality breaks every method in this chapter, yet real data rarely uses all its ambient dimensions. Can we find the directions that carry the most signal and build a lower-dimensional representation around them, so that the clustering methods of Part I regain their footing?

Feedback.

Dimensionality Reduction

Folding and Unfolding Space.

The Why

Dimensionality reduction finds those factors and discards the rest.

Real-world data $\mathbf{X} \in \mathbb{R}^{n \times d}$ is often high-dimensional: images have millions of pixels, genomes have billions of base pairs, text embeddings have thousands of dimensions. Every clustering method in Part I takes such a feature space as given and works within it. But while dimensionality comes at a cost and degrades our approaches, dimensionality carries information. The question then becomes whether a more compact representation could carry the same information.

MNIST handwritten digits , for example, live in $\mathbb{R}^{784}$ (28×28 pixels), but vary along a handful of latent factors such as shapes or geometric primitives. The intrinsic dimensionality is estimated at roughly $k \approx 12$ to 14.

Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791. http://yann.lecun.com/exdb/mnist/

Dimensionality has a price. In low dimensions, some points are close and others are far away; that contrast is what makes clustering work. As d grows, the contrast disappears: all pairwise distances converge toward the same value.

Computational cost grows with d, too. Distance computations scale as $O(nd)$, covariance matrices require $O(d^2)$ storage, and eigendecompositions cost $O(d^3)$. Reducing from d to k before fitting a downstream model can turn an intractable pipeline into a practical one.

Expressiveness of the feature space. Original features are often redundant or entangled. If two features $x^{(i)}$ and $x^{(j)}$ are highly correlated ($|\rho_{ij}| \approx 1$), they carry nearly the same information; a good representation replaces them with a single component that captures their shared variance.

Interpretability benefits directly. Humans usually usually do not think and reason in high-dimensional vector spaces; we prefer

low-dimensional, disentangled representations that align with our concepts of space.

In order to move from high-dimensional observations to compact, signal-carrying representations we have good reason to find ways to,

- find a mapping that retains the pattern that carries the signal.
- construct proxy dimensions that are not limited to the original features, so that hidden patterns become apparent.
- understand what different notions of preserving pattern imply, and choose the right one for a given task.

Hands-On Experience

CONSIDER the canonical six points, but now arranged so that $x^{(1)}$ varies only slightly while $x^{(2)}$ carries nearly all the spread.

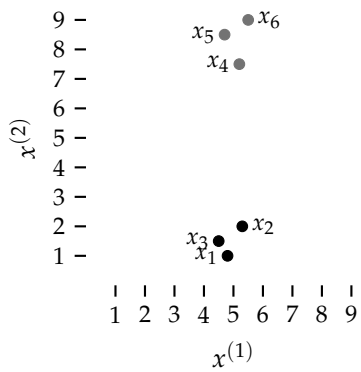


Figure 23: The canonical six points, repositioned so that $x^{(1)}$ barely varies while $x^{(2)}$ carries both the cluster separation and the within-cluster spread. Two features describe each point, but the data is essentially one-dimensional.

THE TWO CLUSTERS are clearly visible, and one feature carries almost all the signal. The first feature $x^{(1)}$ hovers around 5 for every point; dropping it loses almost nothing. The second feature $x^{(2)}$ alone separates the two groups and captures the within-cluster variation.

- If you had to keep only one of the two features, which would you choose?
- How much information would you lose?
- And could you recover the two clusters from that single feature alone?

THE METHODS IN THIS CHAPTER AUTOMATE THAT JUDGMENT OF DIMENSIONALITY REDUCTION. What is the most efficient mapping you can find for the MNIST dataset, using only geometric primitives such as circles, lines so that each class is well-separated?

HAND-CRAFTED FEATURE ENGINEERING is dimensionality reduction by another name. When a domain expert selects "age" and "income" from a database with hundreds of columns, or computes edge gradients from raw pixels, they are projecting from $\mathbb{R}^d$ to $\mathbb{R}^k$ using human genius.

A WELL-CHOSEN MAPPING $f\colon \mathbb{R}^d \to \mathbb{R}^k$ disentangles latent factors, yielding coordinates that are more discriminative for downstream tasks than the originals.

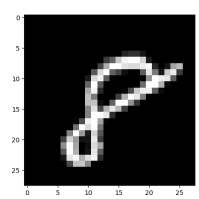


Figure 24: A single MNIST digit its meaningful variation spans few dimensions.

THE LEARNING OBJECTIVES of this lecture:

- Understand naive feature selection based on variance and its limitations.
- Understand the PCA objective and its connection to variance maximization and the singular value decomposition.
- Apply t-SNE and UMAP for visualization and avoid common misinterpretations of their output.
- Choose appropriate dimensionality reduction methods based on linearity, dataset scale, and whether the goal is preprocessing or visualization.

The Curse of Dimensionality, Revisited

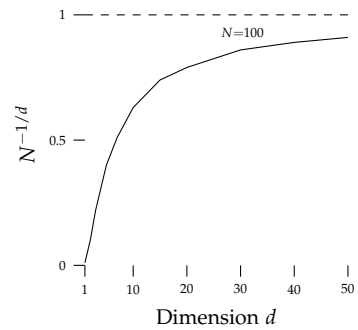


Figure 25: Mean nearest-neighbour spacing $d_{\mathrm{NN}} \propto N^{-1/d}$ for N=100 points. By d=50 every point is nearly as far from its neighbour as from the boundary. But if the data only occupies $k \ll d$ effective dimensions, the curse applies to k, not d.

The key observation is that high d is only fatal when all d dimensions carry independent information. In practice they rarely do. Features are correlated, redundant, or simply noisy. The effective degrees of freedom, the intrinsic dimensionality k, is often far smaller than d.

Reducing d to k reverses the curse. The spacing, the distance concentration, the density collapse all depend on the dimension of the space the data actually lives in. If we project from $\mathbb{R}^d$ down to $\mathbb{R}^k$, the clustering methods from Part I regain their footing: distances become discriminative again, ε-neighbourhoods fill up, and centroids sit among their members.

The blessing of dimensionality. The Johnson-Lindenstrauss lemma shows that n points in high dimensions can be projected to $O(\log n/\epsilon^2)$ dimensions while preserving pairwise distances within $(1\pm\epsilon)$. Even random projections work.

W. B. Johnson and J. Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984. `https://scholar.google.com/scholar?q=Johnson+Lindenstrauss+1984+extensions+lipschitz+mappings+hilbert+space`

The question is how to find the right k directions. Picking original features by hand (as in variance-based selection) works when the axes happen to align with the signal. When they do not, we need methods that construct new directions from the data itself. That is what PCA, and later t-SNE and UMAP, will do.

A Naïve Baseline: Variance-Based Feature Selection

The simplest idea: rank each feature by how much it varies, then drop the ones that barely move.

Before learning any new directions, consider the most straightforward way to reduce dimensions: measure the variance of each original feature independently and keep only those with the highest variance.

Given data $X \in \mathbb{R}^{n\times d}$, compute the variance along each coordinate axis:

$$\mathrm{Var}(x^{(j)}) = \frac{1}{n}\sum_{i=1}^{n}\big(x_i^{(j)} - \bar{x}^{(j)}\big)^2, \quad j = 1, \ldots, d$$

Sort the features by decreasing variance, keep the top k, and discard the rest. This is pure feature selection: no new axes are created, only existing ones are retained or dropped.

When does this work? If the coordinate axes already align with the directions of meaningful variation, variance-based selection is fast, interpretable, and surprisingly effective. A near-constant feature carries almost no information, and removing it loses little.

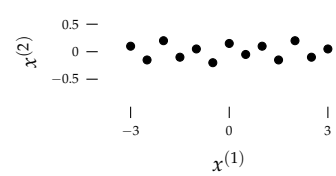


Figure 26: Data spread along $x^{(1)}$ but nearly constant in $x^{(2)}$. Dropping $x^{(2)}$ loses almost nothing.

When does it fail? When the important pattern runs diagonally across the coordinate axes, no single original feature captures it well.

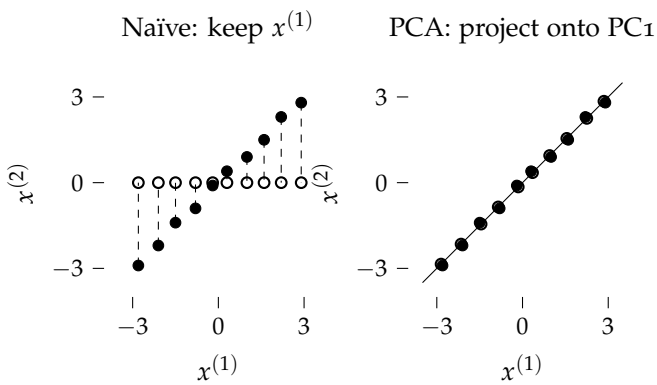


Figure 27: Left: dropping $x^{(2)}$ projects onto the horizontal axis — the dashed errors are large because the data varies along $y{=}x$, not along a coordinate axis. Right: PCA projects onto the direction of maximum variance; the reconstruction errors are tiny. Both features have nearly equal variance, so the naïve ranking cannot decide which to keep.

Here both features have nearly the same variance, so variance-based selection cannot decide which to keep. Dropping either axis incurs large reconstruction error. The real pattern lies along the diagonal $y{=}x$, a direction that no original feature represents.

The lesson: selecting among existing axes is limited to cases where those axes already align with the data. When the interesting variation runs between features, we need to construct new axes. That is exactly what PCA does.

Principal Component Analysis

The manifold hypothesis holds that high-dimensional data concentrates on a k-dimensional manifold $\mathcal{M} \subset \mathbb{R}^d$ with $k \ll d$. The goal of dimensionality reduction is to find a coordinate system for $\mathcal{M}$.

Given centred data $X \in \mathbb{R}^{n \times d}$, where rows are samples, PCA finds a k-dimensional subspace, $k < d$, that best represents the data.

PCA finds orthogonal directions of maximum variance in the data.
K. Pearson. On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11):559–572, 1901. doi: 10.1080/14786440109462720. https://scholar.google.com/scholar?q=Pearson+1901+lines+planes+closest+fit+systems+points; and H. Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441, 1933. doi: 10.1037/h0071325. https://scholar.google.com/scholar?q=Hotelling+1933+analysis+complex+statistical+variables+principal+components

The variance maximization view. Find the direction $w_1 \in \mathbb{R}^d$ with $\|w_1\|{=}1$ that maximises the variance of projected data:

$$\begin{aligned} w_1 &= \arg\max_{\|w\|=1} \operatorname{Var}(Xw) \\ &= \arg\max_{\|w\|=1} w^T \Sigma\, w \end{aligned}$$

where $\Sigma = X^T X / n$ is the covariance matrix.

Lagrange multipliers. Setting $\mathcal{L} = w^T \Sigma w - \lambda(w^T w - 1)$ and differentiating gives $\Sigma w = \lambda w$, so w_1 is the eigenvector of Σ with the largest eigenvalue λ_1.

The solution: w_1 is the eigenvector of Σ corresponding to the largest eigenvalue λ_1. The second principal component is the eigenvector with the second-largest eigenvalue, orthogonal to the first, and

so on. The first k principal components together define the optimal k-dimensional subspace.

THE RECONSTRUCTION ERROR VIEW. Equivalently, PCA minimises the squared reconstruction error:

$$\min_{W \in \mathbb{R}^{d \times k},\, W^T W = I_k} \sum_{i=1}^{n} \|x_i - WW^T x_i\|^2$$

Both views yield the same solution: the first k eigenvectors of the covariance matrix.

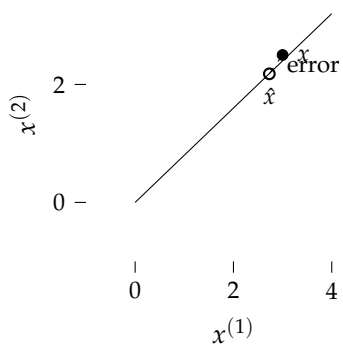


Figure 28: PCA projects x onto the principal subspace (line), minimising the reconstruction error (dashed line to $\hat{x}$).

Algorithm 2: PCA via Eigendecomposition

Require: Centred data $X \in \mathbb{R}^{n \times d}$, target dimension k
Ensure: Principal components $W \in \mathbb{R}^{d \times k}$, projected data $Z \in \mathbb{R}^{n \times k}$
1: Compute covariance matrix $\Sigma = X^T X / n$
2: Compute eigendecomposition $\Sigma = V \Lambda V^T$
3: $W \leftarrow$ first k columns of V (sorted by decreasing eigenvalue)
4: $Z \leftarrow XW$ ▷ Project data onto k-dimensional subspace

A SIMPLE DIAGNOSTIC reveals whether the data is compressible. Inspect the eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_d$ of Σ. If most are near zero, the data varies in only a few directions; the remaining dimensions add volume but no information. The scree plot (Figure 29) visualises exactly this: a steep drop tells you how many components matter.

EXPLAINED VARIANCE. The fraction of total variance explained by the first k components is:

$$\text{explained variance ratio} = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{d} \lambda_i}$$

Choose k so that the cumulative explained variance exceeds a threshold, commonly 95%.

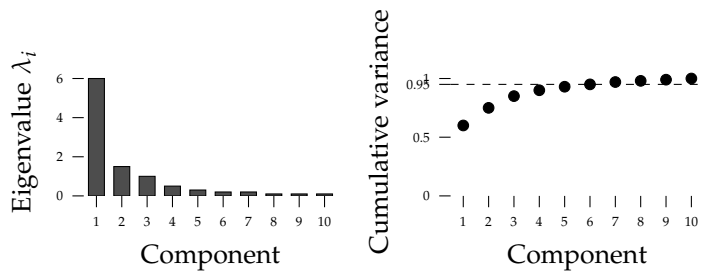


Figure 29: Left: scree plot showing individual eigenvalues; the sharp drop after the first component reveals that one direction dominates. Right: cumulative explained variance; the dashed line marks the 95% threshold, reached at six components.

Singular Value Decomposition

Any matrix $X \in \mathbb{R}^{n \times d}$ has an SVD:

SVD is the workhorse of numerical linear algebra and the computationally preferred route to PCA.

$$X = U \Sigma V^T$$

where $U \in \mathbb{R}^{n \times r}$ holds orthonormal left singular vectors, $\Sigma \in \mathbb{R}^{r \times r}$ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \ldots$, $V \in \mathbb{R}^{d \times r}$ holds orthonormal right singular vectors, and r=rank(X).

Connection to PCA. The columns of V are the eigenvectors of $X^T X$, with $\lambda_i = \sigma_i^2 / n$. In practice, always compute PCA via the SVD: it is numerically stable and avoids forming $X^T X$ explicitly.

Truncated SVD keeps only the top k singular values: $X_k = U_k \Sigma_k V_k^T$. By the Eckart-Young-Mirsky theorem[5], X_k is the best rank-k approximation to X in both Frobenius and spectral norms.

[5] C. Eckart and G. Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936. DOI: 10.1007/BF02288367. https://scholar.google.com/scholar?q=Eckart+Young+1936+approximation+matrix+lower+rank

Independent Component Analysis

PCA finds uncorrelated components. ICA goes further and finds independent components.

ICA seeks statistically independent sources, not just uncorrelated directions.

The cocktail party problem. Multiple microphones record a mixture of speakers. Model: $X = AS$, where A is a mixing matrix and S contains independent source signals. ICA estimates W=A^{-1} to recover S=WX.

The key insight: independent non-Gaussian signals become more Gaussian when mixed, a consequence of the central limit theorem. ICA maximises non-Gaussianity, measured via kurtosis or negentropy, to separate sources. The FastICA[6] algorithm implements this efficiently.

[6] A. Hyvärinen. Fast and robust fixed-point algorithms for independent component analysis. *IEEE Transactions on Neural Networks*, 10(3):626–634, 1999. DOI: 10.1109/72.761722. https://scholar.google.com/scholar?q=Hyvarinen+1999+fast+robust+fixed-point+independent+component+analysis

t-SNE

T-SNE PRESERVES LOCAL PATTERN: nearby points in high dimensions stay nearby in the 2D embedding. It does this in four steps.

T-SNE stands for t-distributed Stochastic Neighbour Embedding . It is optimised for two- or three-dimensional visualization.

Algorithm 3: t-SNE

Require: High-dimensional data X, perplexity, target dimension k

1: Compute pairwise similarities in high-D (Gaussian kernel): $p_{j|i} = \exp(-\|x_i - x_j\|^2/2\sigma_i^2)/\sum_{l\neq i}\exp(-\|x_i - x_l\|^2/2\sigma_i^2)$; symmetrize: $p_{ij} = (p_{j|i} + p_{i|j})/2n$
2: Initialize low-D embedding Y
3: **repeat**
4: Compute low-D similarities with Student-t kernel: $q_{ij} = (1 + \|y_i - y_j\|^2)^{-1}/\sum_{k\neq l}(1 + \|y_k - y_l\|^2)^{-1}$
5: Gradient step: minimise $\mathrm{KL}(P\|Q) = \sum_{ij} p_{ij}\log(p_{ij}/q_{ij})$
6: **until** convergence

L. van der Maaten and G. Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008. https://scholar.google.com/scholar?q=van+der+Maaten+Hinton+2008+visualizing+data+t-SNE

PERPLEXITY controls the effective number of neighbours each point considers. It sets the bandwidth σ_i of the Gaussian kernel. Typical values: 5–50. Higher perplexity means more global pattern is considered.

PERPLEXITY is defined as $2^{H(P_i)}$ where H is the Shannon entropy of the conditional distribution over neighbours. Think of it as the effective number of neighbours.

INTERPRETING T-SNE OUTPUT requires caution.

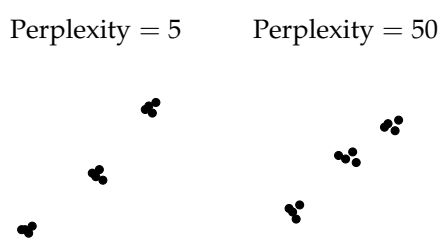


Figure 30: Same data at two perplexity values. Cluster shapes, sizes, and between-cluster distances all change, neither reflects original geometry.

COMMON PITFALLS:

- Cluster sizes are meaningless: t-SNE equalises local densities.
- Distances between clusters are meaningless: the embedding preserves local, not global, pattern.
- Results vary with random initialisation; run multiple times.
- t-SNE is for visualization only, not for preprocessing or downstream tasks.

UMAP

UMAP IS A NEWER NON-LINEAR METHOD that typically preserves more global pattern than t-SNE, runs faster on large datasets, and has a theoretical foundation in manifold learning.

UMAP stands for Uniform Manifold Approximation and Projection . It has theoretical foundations in Riemannian geometry and algebraic topology.

L. McInnes, J. Healy, and J. Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018. https://arxiv.org/abs/1802.03426

KEY IDEAS.

1. Build a weighted k-nearest-neighbour graph in high dimensions.
2. Find a low-dimensional embedding that preserves the graph topology.
3. Use cross-entropy loss, rather than KL divergence, to match fuzzy simplicial sets.

Algorithm 4: UMAP

Require: High-dimensional data X, number of neighbours k, target dimension d, minimum distance δ

For each x_i, find its k nearest neighbours; set $\rho_i = \min_{j \in \mathrm{NN}(i)} \|x_i - x_j\|$

Choose σ_i so that $\sum_{j \in \mathrm{NN}(i)} \exp\big(-(\|x_i - x_j\| - \rho_i)/\sigma_i\big) = \log_2 k$

Compute directed edge weights: $w_{j|i} = \exp\big(-\max(0, \|x_i - x_j\| - \rho_i)/\sigma_i\big)$

Symmetrise: $w_{ij} = w_{j|i} + w_{i|j} - w_{j|i} \cdot w_{i|j}$ ▷ Fuzzy union of local neighbourhoods

Initialise low-dimensional coordinates Y (e.g. via spectral embedding)

repeat

  Compute low-D weights: $v_{ij} = \big(1 + a\,\|y_i - y_j\|^{2b}\big)^{-1}$, where a, b are derived from δ

  Gradient step: minimise fuzzy set cross-entropy $\mathcal{L} = \sum_{ij} \big[w_{ij} \log \frac{w_{ij}}{v_{ij}} + (1 - w_{ij}) \log \frac{1 - w_{ij}}{1 - v_{ij}}\big]$

until convergence

KEY PARAMETERS. `n_neighbors`, similar to perplexity and typically set between 15 and 200, controls how much local versus global pattern is preserved. `min_dist`, typically 0.1 to 0.5, controls how tightly points cluster in the embedding.

CHOOSING A METHOD depends on what you want to preserve and what comes next.

Method	Linear	Scalable	Best use
PCA	Yes	Yes	Preprocessing, denoising, linear pattern
ICA	Yes	Yes	Source separation, statistical independence
t-SNE	No	Medium	Visualization of small datasets ($n < 10\,000$)
UMAP	No	Yes	Visualization of large datasets, more global pattern

Table 2: Comparison of dimensionality reduction methods. None of the non-linear methods are appropriate for preprocessing features for downstream learning.

Examples & Exercises

COMPUTING BY HAND builds the geometric intuition that plotting the output never can. Step through the arithmetic slowly.

GIVEN CENTRED DATA $X = \begin{pmatrix} 2 & 1 \\ 1 & 2 \\ -1 & -2 \\ -2 & -1 \end{pmatrix}$, compute the covariance matrix Σ, find its eigenvalues and eigenvectors, and determine what fraction of variance PC1 explains.

The covariance matrix:

$$\begin{aligned} \Sigma &= \frac{1}{4} X^T X \\ &= \frac{1}{4} \begin{pmatrix} 10 & 8 \\ 8 & 10 \end{pmatrix} \\ &= \begin{pmatrix} 2.5 & 2 \\ 2 & 2.5 \end{pmatrix} \end{aligned}$$

Eigenvalues from $\det(\Sigma - \lambda I) = 0$:

$$\begin{aligned} (2.5 - \lambda)^2 - 4 &= 0 \\ \lambda_1 &= 4.5 \\ \lambda_2 &= 0.5 \end{aligned}$$

Eigenvectors: $v_1 = (1, 1)/\sqrt{2}$, $v_2 = (1, -1)/\sqrt{2}$. Explained variance by PC1:

$$\begin{aligned} \frac{\lambda_1}{\lambda_1 + \lambda_2} &= \frac{4.5}{5} \\ &= 90\% \end{aligned}$$

Now derive the projection of each data point onto PC1 yourself, then verify that the projected points have variance 4.5.

CHOOSING A THRESHOLD. A dataset has eigenvalues: 100, 50, 20, 10, 5, 3, 2, 1, 0.5, 0.5 (total =192). How many components explain 90% of the variance? How many for 99%? Sketch the cumulative explained variance curve and identify the elbow.

Cumulative sums: 100, 150, 170, 180, 185, 188, 190, 191, 191.5, 192.

90% threshold: $\geq$ 172.8, so 3 components.
99% threshold: $\geq$ 190.1, so 7 components.

T-SNE INTERPRETATION TRAP. You run t-SNE on a dataset and observe: cluster A has diameter 10 units, cluster B has diameter 5 units, and the distance between clusters is 20 units. What can you conclude? What can you not conclude?

Think carefully: t-SNE preserves local neighbourhood, not global geometry. Cluster sizes and between-cluster distances in the 2D plot tell you nothing about the original space.

CODE will be provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

A COFFEE AROMA DATASET TO EXPLORE. You are given 200 aroma fingerprints in 50 dimensions, four brewing varieties. Apply PCA, retaining enough components to explain 95% of variance. Scatterplot the first two principal components, colour-coded by variety. Run t-SNE and UMAP on the same data and compare the three embeddings.

WHAT TO OBSERVE. PCA preserves global pattern: the four varieties form elongated regions that reflect their actual distances in the original space. t-SNE compresses each variety into a tight ball; distances between balls are arbitrary. UMAP may show clearer cluster separation than t-SNE while better preserving global topology. Reconstruction error from PCA tells you whether 50 dimensions were truly needed.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- What does it mean for data to have low intrinsic dimensionality?
- Why does PCA correspond to an eigenvector decomposition of the covariance matrix?
- What is the difference between an uncorrelated and an independent component?
- Why does t-SNE use a Student-t kernel in the low-dimensional space but a Gaussian in the high-dimensional space?

- When would you prefer UMAP over t-SNE, and when would you use PCA instead of either?
- What does the perplexity parameter actually control, and what happens when it is too low or too high?
- Why should you never use t-SNE or UMAP embeddings as features for a downstream classifier?
- How does the curse of dimensionality motivate dimensionality reduction before running K-Means?

RECAP of Key Concepts:

- PCA finds the orthogonal directions of maximum variance via eigendecomposition of the covariance matrix; SVD is the numerically preferred implementation.
- The explained-variance ratio guides the choice of k; retain enough components to exceed a threshold such as 95%.
- t-SNE and UMAP preserve local neighbourhood patterns for visualization; cluster sizes and between-cluster distances in their output are not interpretable.
- ICA finds statistically independent sources by maximising non-Gaussianity, going beyond the uncorrelated directions found by PCA.
- No non-linear method should be used as a preprocessing step for downstream learning; use PCA or SVD for that purpose.

PCA COMPRESSES EACH INPUT TO A SINGLE POINT. Given two coffee aroma profiles that differ only in a small measurement noise, their PCA projections land at two nearby points with no indication of how reliably they were placed there. There is no principled way to generate a new profile by sampling from the compressed space, and no way to interpolate smoothly between two known profiles.

THE FUNDAMENTAL LIMIT of every method in this chapter: each input is mapped to a single point. There is no way to say how uncertain that mapping is, or to generate new plausible inputs by sampling from the compressed space.

VARIATIONAL AUTOENCODERS replace the single point with a distribution. The encoder no longer outputs coordinates but outputs a mean and a variance, turning compression into probabilistic inference. Every point in the latent space corresponds to something meaningful, and sampling from it generates new, plausible data.

TEASER. Can we replace the single compressed point with a distribution, so that sampling from the latent space generates new, plausible data?

FEEDBACK

Index

border point, 15
bridging, 20

chaining
  DBSCAN, 20
cluster shape, 13
computational complexity
  DBSCAN, 16
convergence, 16
core point, 15
covariance matrix, 33
curse of dimensionality, 21, 29, 32

DBSCAN, 13, 15
  parameter selection, 19
density connectivity, 15
density reachability, 15
density-based clustering, 13
determinism, 16
dimensionality reduction, 29

epsilon-neighborhood, 15
examples, 23, 38
exercises, 23, 38

failure modes
  DBSCAN, 20
feature selection, 32

geometric assumptions, 13

HDBSCAN, 21
high dimensionality, 21
hyperparameter selection, 19

ICA, 35
initialization
  DBSCAN, 16
intrinsic dimensionality, 29

k-distance plot, 19

license, 2

noise, 13
noise point, 15
non-convex clusters, 13

OPTICS, 20
outliers, 13

partition, 15
PCA, 29, 33

recap, 27, 39
reflection, 27, 39

statistical independence, 35
SVD, 35

t-SNE, 29, 36

UMAP, 29, 37

variance, 32